Projet
Programmation Orientée Objet en C++
ALTERDUNE

Daniel Wladdimiro
ESILV
daniel.wladdimiro@devinci.fr

Second semestre 2025-2026

1 Introduction

La programmation orientée objet (POO) est un paradigme fondamental en informatique, largement utilisé pour le développement de logiciels bien structurés et modulaires.

Ce projet doit être réalisé en **binôme** (équipes de deux). Il propose une approche ludique et pédagogique à travers la création d'un mini-jeu de type RPG nommé **ALTERDUNE**. L'univers est particulièrement adapté à la POO : le joueur, les monstres, les objets, les actions et les combats peuvent être modélisés sous forme d'objets, organisés en classes et en hiérarchies simples.

2 Description du Projet

Le projet consiste à développer un jeu par console, structuré autour :
— un **menu principal** (hors combat),
— des **combats au tour par tour**,
— un système de **fins multiples** selon les choix du joueur,
— le **chargement de données** depuis **deux fichiers obligatoires**.

2.1 Objectif du projet

L'objectif est de mettre en pratique :
— l'**encapsulation** (attributs privés, accesseurs/modificateurs),
— l'**héritage** (joueur/monstres),
— le **polymorphisme** (comportements différents selon la catégorie de monstre),
— la **composition** (inventaire, actions ACT, bestiaire),
— la **lecture de fichiers** pour initialiser les objets du jeu.

3 Démarrage d'une partie

Au lancement du programme :

1. Le joueur saisit le **nom de son personnage**.
2. Le jeu charge **deux fichiers** :
   — `items.csv` : items utilisables par le joueur (inventaire initial).
   — `monsters.csv` : ennemis possibles, utilisés pour générer des combats aléatoires.
3. Le jeu affiche un résumé :
   — le **nom** du joueur,
   — les **HP** du joueur (HP actuel et HP maximum),
   — la **liste des items** disponibles (nom + quantité).

4 Condition de fin de partie et fins multiples

Une partie se termine lorsque le joueur obtient **10 victoires** (10 combats gagnés).
À la fin, le jeu affiche une fin selon l'historique :
— **Fin Génocidaire** : le joueur a **tué** tous les monstres vaincus (0 épargné).
— **Fin Pacifiste** : le joueur a **épargné** tous les monstres vaincus (0 tué).
— **Fin Neutre** : le joueur a à la fois **tué** et **épargné**.

5 Monstres, catégories et actions ACT

5.1 Catégories de monstres

Chaque monstre appartient à une catégorie, qui détermine le nombre d'actions ACT disponibles :
— **NORMAL** : 2 actions ACT possibles
— **MINIBOSS** : 3 actions ACT possibles
— **BOSS** : 4 actions ACT possibles

5.2 Système Mercy

Chaque monstre possède une jauge **Mercy** (par exemple de 0 à 100). Certaines actions ACT font évoluer cette jauge. Si **Mercy** ≥ 100, le joueur peut utiliser **MERCY** pour **épargner** le monstre et gagner le combat sans le tuer.

5.3 Catalogue d'actions ACT (pré-défini)

Les actions ACT sont **pré-définies dans le jeu** (dans le code C++), sous la forme d'un catalogue (par exemple un `map` ou un `vector`). Une action ACT possède au minimum :
— un **identifiant** (ex. `JOKE`, `COMPLIMENT`, `INSULT`),
— un **texte affiché** (un texte drôle/absurde, affiché lorsqu'on exécute l'action),
— un **impact sur Mercy** : un entier **positif** (le monstre s'apaise), **négatif** (le monstre s'énerve) ou éventuellement 0.

Contraintes :
— Le catalogue doit contenir **au minimum 8 actions** différentes.
— Il doit exister **au minimum 2 actions** avec un impact **négatif** sur Mercy (actions qui énervent).

— Lorsque Mercy est modifié, la valeur est **bornée** : Mercy ne peut pas descendre sous 0, ni dépasser la valeur objectif (par exemple 100).

Remarque : le fichier `monsters.csv` ne définit pas le texte des actions. Il indique uniquement quelles actions (identifiants) sont disponibles pour chaque monstre. Le comportement (texte + variation de Mercy) est fourni par le catalogue dans le code.

6 Menu de Simulation (menu principal)

Pour permettre aux utilisateurs d'interagir avec le jeu, un menu sera mis en place avec les options suivantes :

— **Bestiaire (bitácora de monstres)**
— **Démarrer un combat**
— **Statistiques du personnage**
— **Items**
— **Quitter**

6.1 Bestiaire (bitácora de monstres)

Le bestiaire affiche **tous les monstres vaincus** (combats gagnés), avec leurs caractéristiques :

— Nom
— Catégorie : `NORMAL`, `MINIBOSS`, `BOSS`
— Statistiques (HP max, attaque, défense)
— Résultat du combat : **Tué** ou **Épargné**

6.2 Statistiques du personnage

Affiche au minimum :

— Nom du joueur
— HP actuel et HP maximum
— Nombre de monstres **tués**
— Nombre de monstres **épargnés**
— Nombre de **victoires** (0 à 10)

6.3 Items

Affiche l'inventaire du joueur :

— Liste des items disponibles et leurs quantités
— Description simple (ex. : soigne X HP)
— Possibilité d'**utiliser** un item (réduction de quantité + application de l'effet)

L'utilisation d'un item depuis ce menu (hors combat) est autorisée.

6.4 Démarrer un combat

Lance un combat contre un ennemi **tiré aléatoirement** depuis la liste chargée depuis `monsters.csv`.

7 Combat

Le combat oppose le joueur à un monstre. Tant que les deux sont vivants, le tour se répète.

7.1 Menu de combat

À chaque tour du joueur, afficher le menu :

FIGHT **ACT** **ITEM** **MERCY**

7.2 Règles de dégâts

Pour le porjet, les dégâts sont calculés à partir d'un tirage aléatoire.
À chaque attaque, on tire un entier aléatoire :

$$\textit{dégâts} = \textit{rand}\left(0,\ HP^{\max}_{\textit{défenseur}}\right)$$

où $HP^{\max}_{\textit{défenseur}}$ est la vie maximale de la cible (monstre ou joueur).
Interprétation :
— Si $\textit{dégâts} = 0$, l'attaque n'inflige rien (coup raté).
— Sinon, les HP du défenseur diminuent de *dégâts*.
— Les HP ne peuvent pas descendre sous 0.
— Si les HP atteignent 0, l'entité est considérée comme vaincue.
Remarque d'implémentation : l'aléatoire devra être géré proprement (par exemple avec `<random>`), et le tirage doit être refait à chaque attaque.

7.3 Actions du joueur

7.3.1 FIGHT

Le joueur attaque directement. Si le monstre atteint 0 HP, il est **tué**, le combat est gagné et la victoire augmente de 1.

7.3.2 ACT

Le joueur choisit une action parmi celles disponibles pour ce monstre (2/3/4 selon la catégorie). Quand une action est choisie :
— un **texte drôle** (pré-défini) est affiché,
— la jauge **Mercy** est modifiée (augmentation ou diminution possible),
— la valeur de Mercy reste bornée (entre 0 et Mercy objectif).

7.3.3 ITEM

Le joueur utilise un item de l'inventaire (souvent pour se soigner). Cela consomme le tour.

7.3.4 MERCY

Le joueur peut **épargner** le monstre si **Mercy** ≥ 100. Le combat est alors gagné et la victoire augmente de 1 (monstre **épargné**, non tué).

7.4 Tour du monstre

Si le monstre est encore vivant après l'action du joueur, il attaque le joueur (même formule). Si le joueur atteint 0 HP, la partie est perdue (défaite immédiate).

8 Chargement des Données

Les données du jeu seront chargées à partir de fichiers `csv` afin d'initialiser les objets des différentes classes du programme.

Le jeu doit obligatoirement charger **deux fichiers** au démarrage :

— **`items.csv`** : inventaire initial du joueur.
— **`monsters.csv`** : base d'ennemis utilisés pour tirer des combats aléatoires.

Les étudiants doivent gérer au minimum :

— l'erreur « fichier introuvable » (message + arrêt propre),
— l'erreur « ligne mal formée » (ignorer la ligne ou arrêter avec message, au choix).

8.1 `items.csv`

Format recommandé (séparateur `;`) :

`nom;type;valeur;quantite`

Contraintes :

— `type` peut être limité à `HEAL` (soin) pour rester simple.
— `valeur` : nombre de HP rendus.
— `quantite` : quantité initiale disponible.

Exemple :

```
Potion;HEAL;15;3
Snack;HEAL;8;5
SuperPotion;HEAL;30;1
```

8.2 `monsters.csv`

Chaque monstre comprend :

— Catégorie : `NORMAL` / `MINIBOSS` / `BOSS`
— Nom
— Statistiques : HP, ATK, DEF
— Mercy objectif (recommandé : 100)
— Liste des **identifiants** d'actions ACT (pré-définies dans le jeu)

Format recommandé (séparateur `;`) :

`categorie;nom;hp;atk;def;mercyGoal;act1;act2;act3;act4`

Règles :

— **NORMAL** utilise seulement `act1, act2` (les champs `act3, act4` peuvent valoir `-`).
— **MINIBOSS** utilise `act1, act2, act3` (le champ `act4` peut valoir `-`).
— **BOSS** utilise `act1, act2, act3, act4`.
— Les identifiants doivent exister dans le **catalogue d'actions ACT** du code.

Exemples :

```
NORMAL;Froggit;30;7;1;100;COMPLIMENT;DISCUSS;-;-
MINIBOSS;MimicBox;45;10;2;100;OBSERVE;PET;OFFER_SNACK;-
BOSS;QueenByte;80;15;4;100;REASON;DANCE;JOKE;INSULT
```

9 Évaluation

Le projet sera évalué en **trois temps**, répartis sur plusieurs séances de TD. La réalisation en **binôme** est **obligatoire**.

9.1 TD11–TD12 : Mini-suivi (20% de la note finale)

Lors du mini-suivi, chaque binôme doit présenter le **UML initial** et expliquer les **avancées déjà réalisées**. L'objectif principal est de vérifier un bon démarrage et une progression claire.

Le mini-suivi est noté sur *20 points* (puis pondéré à 20%) :

- **UML initial** (8 points) : diagramme lisible, cohérent avec l'idée du projet.
- **Avancées techniques** (8 points) : premières classes, structure `.h/.cpp`, début des menus et/ou lecture de fichiers, premiers tests.
- **Clarté des explications** (4 points) : capacité à justifier les choix (rôles des classes, responsabilités, relations).

9.2 TD15–TD16 : Soutenance – Partie 1 (80% de la note finale)

Chaque binôme réalise une soutenance de **15 minutes** (format libre : slides, vidéo, démonstration directe, etc.). La soutenance doit inclure :

- présentation du **UML final** ;
- explication de **toutes les fonctionnalités** implémentées ;
- **démonstration (demo)** du jeu.

La soutenance (Partie 1) est notée sur *20 points* (puis pondérée à 80%) :

- **Diagramme UML** (2 points) : UML final clair, structuré et à jour.
- **Héritage** (2 points) : structuration correcte des classes (joueur/monstres/catégories).
  - Classe abstraite (1 point).
  - Classes dérivées (1 point).
- **Encapsulation** (2 points) : attributs privés + accesseurs/modificateurs pertinents.
- **Polymorphisme** (2 points) : comportement dépendant de la catégorie (2/3/4 actions ACT, etc.).
- **Système ACT et Mercy** (2 points) : catalogue d'actions pré-défini (textes drôles) + effets sur Mercy (positifs et négatifs) + bornage de Mercy.
- **Lecture des fichiers** (2 points) : chargement de `items.csv` et `monsters.csv` (parsing + erreurs minimales).
- **Implémentation du jeu** (8 points) :
  - Menu principal complet (bestiaire, stats, items, combat) (3 points).
  - Système de combat (FIGHT/ACT/ITEM/MERCY + tour monstre) (3 points).
  - Condition de fin à 10 victoires + fins multiples (génocidaire/neutre/pacifiste) (2 points).

— **Présentation** (2 points) : clarté, démonstration, explication des choix techniques.

9.3 TD17–TD18 : Soutenance – Partie 2 (Questions/Réponses)

Cette seconde partie consiste en une session de **questions/réponses** avec l'enseignant, afin d'évaluer la **maîtrise** du projet par chaque membre du binôme (code, choix techniques, POO, fichiers, aléatoire, gestion des cas limites, etc.).

Important : cette partie peut entraîner une **pénalisation importante** si l'enseignant considère qu'il n'y a pas de maîtrise suffisante (par exemple : incapacité à expliquer le code, confusion sur l'architecture, ou réponses incorrectes).

9.4 Bonus (projet « sur-salient »)

Si le projet est considéré comme **sur-salient** (au-delà de ce qui est demandé, sans complexifier inutilement), un **bonus** peut être accordé, jusqu'à **+4 points** sur la note finale.

9.5 Formule de note finale

La note finale est calculée ainsi :

$$\textit{Note finale} = 0{,}2 \times \textit{Note mini-suivi} + 0{,}8 \times \textit{Note soutenance (Partie 1)} + \textit{bonus} - \textit{pénalité}$$